

NFCU DataPower OpenShift Deployment

Lab Guide

Version: 3.2.4

Last Updated: July 9, 2026

Lab Format: Hands-on

Contents

Lab goal	2
Learning outcomes	2
Lab inputs.....	2
Getting your OpenShift login command	3
Files used in this lab.....	3
Lab 1: Log in to OpenShift	4
Prerequisites	4
Objective	5
Lab 2: Create or select the project	6
Objective	6
Lab 3: Install the DataPower Operator.....	7
Objective	7
Lab 4: Prepare secrets and ConfigMaps	8
Objective	8
Lab 5: Deploy DataPower	10
Objective	10
Lab 6: Expose the web UI and MPG	11
Objective	11
Lab 7: Validate the deployment	14
Objective	14
Lab 8: Pull configuration from Git	16
Objective	16
Troubleshooting.....	22
Lab 9: Configure Persistent Storage for IBM DataPower on OpenShift.....	24
Objective	24
Lab Overview	24

Lab goal

In this lab, students deploy IBM DataPower Gateway on OpenShift, expose the web management interface and a Multi-Protocol Gateway route, validate the deployment, and extend the deployment with a Git-based configuration workflow.

Learning outcomes

By the end of this lab, students should be able to:

1. Log in to an OpenShift cluster.
2. Create or select the target project.
3. Install the IBM DataPower Operator.
4. Create the secrets and ConfigMaps required for a base deployment.
5. Deploy a DataPowerService instance.
6. Expose the Web UI (for POT only) and MPG endpoint.
7. Validate that the platform is working.
8. Update the deployment to pull configuration from Git.

Lab inputs

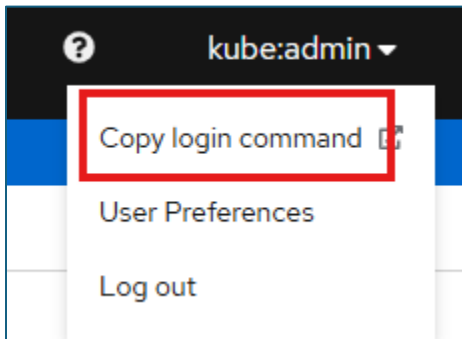
Before you begin, collect the following information from your instructor or lab environment:

Item	Value
OpenShift console URL	https://console-openshift-console.apps.<cluster-name>.<domain>
OpenShift API endpoint	https://api.<cluster-name>:6443
OpenShift username	kubeadmin or the username provided to you
OpenShift password or token	Provided by your instructor or administrator
IBM entitlement key	Provided before the lab starts

Email for entitlement secret	Your email address
Cluster apps domain	Determine during the lab if needed
Github repo	https://github.com/innessrg/NFCU_DataPower_MVP1

Getting your OpenShift login command

- In the OpenShift console, open the user menu and select **Copy login command**.



- Select **Display Token**.

Example token login command:

Command:

```
oc login --token=<token> --server=https://api.<cluster-name>:6443
```

Files used in this lab

Download the lab files from the GitHub repository before starting: [NFCU_DataPower_MVP1](https://github.com/innessrg/NFCU_DataPower_MVP1).

Files referenced in this lab include:

- Datapower.yaml
- web-mgmt.cfg
- webgui-service.yaml
- webgui-route.yaml
- route.yaml
- service.yaml
- pull-from-git.cfg

- `github.com.cer`
- `pot-additionalDomainConfig.yaml`

Lab 1: Log in to OpenShift

Prerequisites

Complete the following before the lab begins:

- Access to an OpenShift cluster.
- The `oc` CLI installed and working.
- The IBM entitlement key.
- Access to the GitHub lab repository.
- Permission to create resources in the target namespace.

Objective

Authenticate the cluster and confirm that CLI access works.

Steps

1. Get your login command from the OpenShift console, or use the username and password provided by your instructor.
2. Run the login command.

Command:

```
oc login --token=<token> --server=https://api.<cluster-name>:6443
```

3. Confirm that authentication succeeded.

Command:

```
oc whoami
```

```
oc cluster-info
```

4. Record the current user and cluster details for your lab notes.

Checkpoint

- `oc whoami` returns your username.
- `oc cluster-info` returns cluster details.

Lab 2: Create or select the project

Objective

Use the target namespace for the operator and DataPower deployment.

Steps

1. Create the project if it does not already exist.

Command:

```
oc new-project nfcu-demo
```

2. If the project already exists, switch to it.

Command:

```
oc project nfcu-demo
```

3. Confirm the active project.

Command:

```
oc project
```

Checkpoint

- The current project is nfcu-demo.

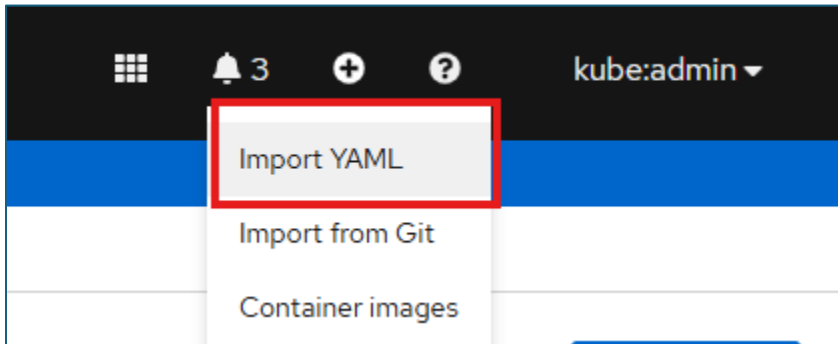
Lab 3: Install the DataPower Operator

Objective

Install the catalog source and the IBM DataPower Operator.

Steps

1. Open **datapower-catalog-source-latest.yaml** and copy its contents.
2. In the OpenShift console, Click the “+” to open **Import YAML** and paste the file contents.



3. Set the namespace to **openshift-marketplace**, then create the resource.
4. Review the created catalog source in the console and confirm it was applied successfully.
5. Go to **Operators > OperatorHub** and search for **IBM DataPower Gateway**.
6. Select the operator and click **Install**.
7. Choose the installation mode, update channel, and approval strategy required for your lab environment.
8. Click **Subscribe** and wait for the installation to complete.
9. Open **Operators > Installed Operators** and confirm the operator appears without errors.

Checkpoint

- The subscription exists.
- The DataPower ClusterServiceVersion is installed.
- The operator pod is running.

Lab 4: Prepare secrets and ConfigMaps

Objective

Create the resources required for the base DataPower deployment.

Steps

1. Confirm that you are in the `nfcu-demo` project.

Command:

```
oc project nfcu-demo
```

2. Create the admin password secret.

Command:

```
oc create secret generic dp-admin-password --from-literal=password=admin -n nfcu-demo
```

3. Create the GitHub certificate secret referenced by `Datapower.yaml`.

Command:

```
oc create secret generic github.com.cer --from-file=github.com.cer -n nfcu-demo
```

4. Create the IBM entitlement secret so the operator can pull the DataPower image from the IBM Container Registry.

Command:

```
oc create secret docker-registry ibm-entitlement-key --docker-server=cp.icr.io --docker-username=cp --docker-password=<YOUR_IBM_ENTITLEMENT_KEY> --docker-email=<YOUR_EMAIL> -n nfcu-demo
```

5. Link the entitlement secret to the default service account for image pulls.

Command:

```
oc secrets link default ibm-entitlement-key --for=pull -n nfcu-demo
```

6. Create the web management ConfigMap from `web-mgmt.cfg`.

Command:

```
oc create configmap web-mgmt-cfg --from-file=web-mgmt.cfg -n nfcu-demo
```

7. Verify that the required resources exist.

Command:

```
oc get configmap web-mgmt-cfg -n nfcu-demo  
oc get secret dp-admin-password -n nfcu-demo  
oc get secret github.com.cer -n nfcu-demo  
oc get secret ibm-entitlement-key -n nfcu-demo
```

Checkpoint

- web-mgmt-cfg exists.
- dp-admin-password exists.
- github.com.cer exists.
- ibm-entitlement-key exists.
- The entitlement key is linked to the default service account.

Lab 5: Deploy DataPower

Objective

Create the DataPowerService resource and wait for it to become ready.

Steps

1. Review `Datapower.yaml` and confirm the namespace, version, deployment name, and domain configuration are correct for your environment.
2. In the OpenShift console, go to **Operators > Installed Operators**.
3. Open **IBM DataPower Gateway**.
4. Under **Provided APIs**, select **DataPowerService**.
5. Click **Create Instance**.
6. In **Form view**, review the key fields such as name, namespace, version, users, domains, and replicas.
7. Switch to **YAML view**, paste the contents of `Datapower.yaml`, and click **Create**.
8. Wait for the DataPowerService status to reach **Running**.
9. Optionally monitor progress from the CLI.

Command:

```
oc get pods -w
oc get datapowerservice quickstart
oc logs quickstart-0
```

Checkpoint

- The pod reaches `Running`.
- The DataPowerService becomes ready.
- Base startup completes successfully.

Lab 6: Expose the web UI and MPG

Objective

Create the services and routes required to access the web UI and the Multi-Protocol Gateway.

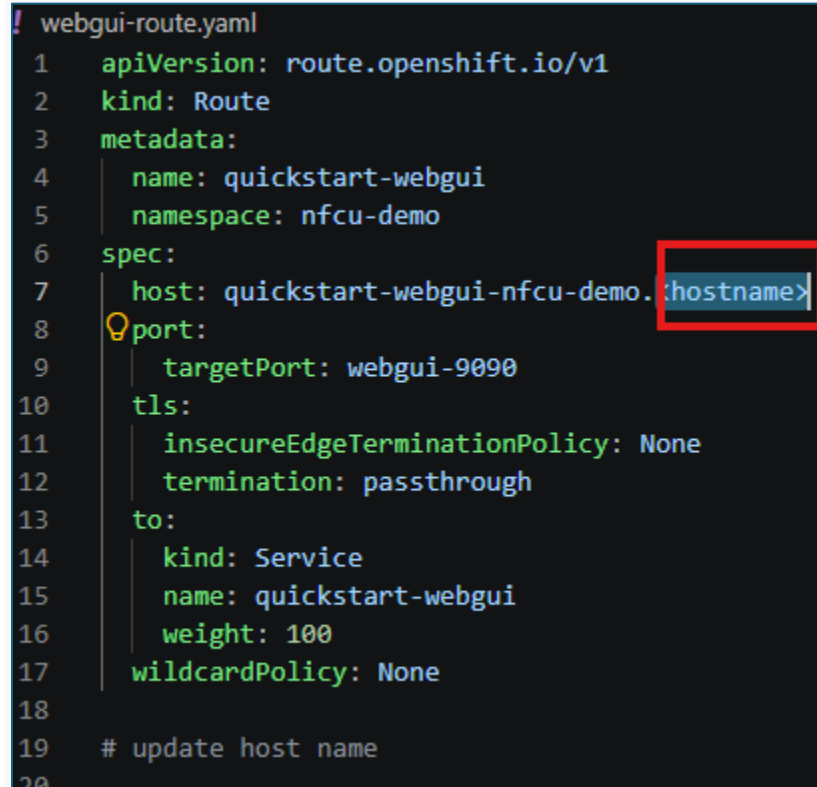
Steps

1. Open `webgui-route.yaml`.
2. Update `spec.host` field using the pattern `<route-or-service-name>-<namespace>.<apps-domain>`.
3. If needed, determine the cluster apps domain.
- 4.

Command:

```
oc get ingresses.config/cluster -o jsonpath='{.spec.domain}'
```

Copy output of command in the `<hostname>`



```
! webgui-route.yaml
1  apiVersion: route.openshift.io/v1
2  kind: Route
3  metadata:
4    name: quickstart-webgui
5    namespace: nfcu-demo
6  spec:
7    host: quickstart-webgui-nfcu-demo.<hostname>
8    port:
9      targetPort: webgui-9090
10   tls:
11     insecureEdgeTerminationPolicy: None
12     termination: passthrough
13   to:
14     kind: Service
15     name: quickstart-webgui
16     weight: 100
17   wildcardPolicy: None
18
19 # update host name
20
```

4. Save `webgui-route.yaml` after updating the host name.
5. Apply the web UI service.

Command:

```
oc apply -f webgui-service.yaml
```

6. Apply the web UI route.

Command:

```
oc apply -f webgui-route.yaml
```

7. Apply the MPG service.

Command:

```
oc apply -f service.yaml
```

8. Apply the MPG route.

Command:

```
oc apply -f route.yaml
```

9. List the routes.

Command:

```
oc get route
```

10. Get the web UI route hostname.

Command:

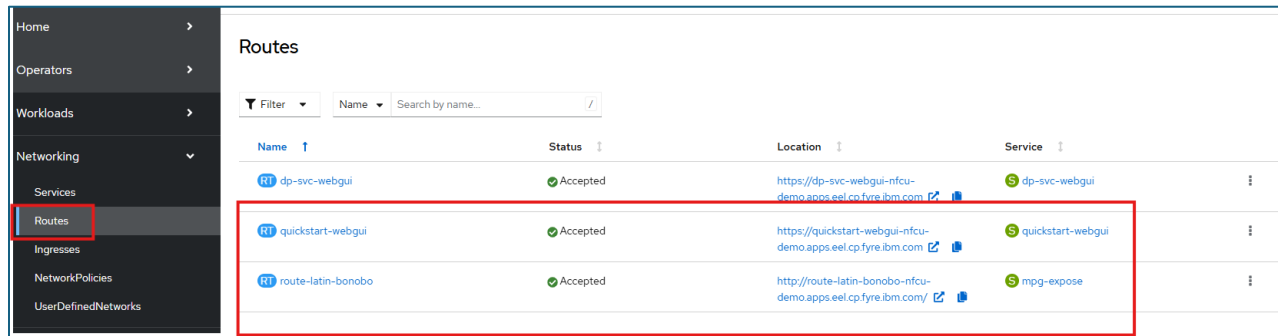
```
oc get route quickstart-webgui -o jsonpath='{.spec.host}'
```

11. Get the MPG route hostname.

Command:

```
oc get route route-latin-bonobo -o jsonpath='{.spec.host}'
```

Note: You can also Validate in the Openshift Console – Networking → Routes



Routes				
Filter Name Search by name... /				
Name ↑	Status ↓	Location ↓	Service ↓	
 dp-svc-webgui	 Accepted	https://dp-svc-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com/  	 dp-svc-webgui	
 quickstart-webgui	 Accepted	https://quickstart-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com/  	 quickstart-webgui	
 route-latin-bonobo	 Accepted	http://route-latin-bonobo-nfcu-demo.apps.eel.cp.fyre.ibm.com/  	 mpg-expose	

Checkpoint

- The web UI route exists.
- The MPG route exists.

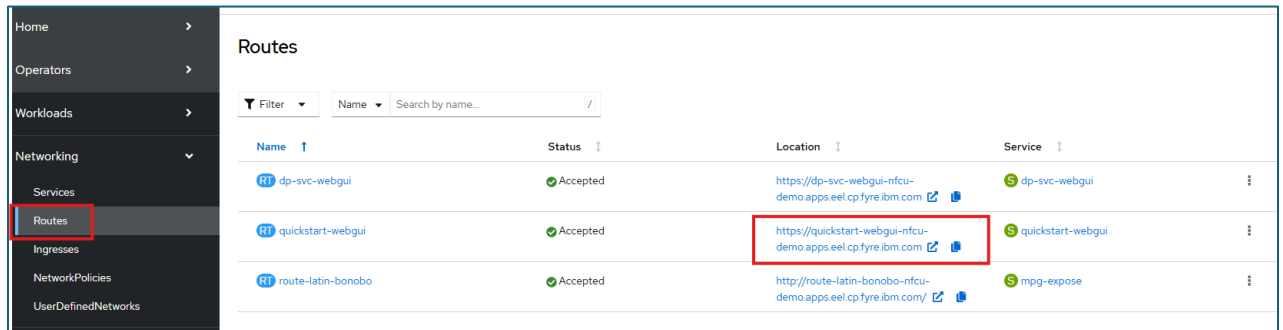
Lab 7: Validate the deployment

Objective

Confirm that the deployment works from both the administrative and API perspectives.

Steps

1. Open the DataPower web UI in a browser using the **quickstart-webgui** route hostname or click on the Location:

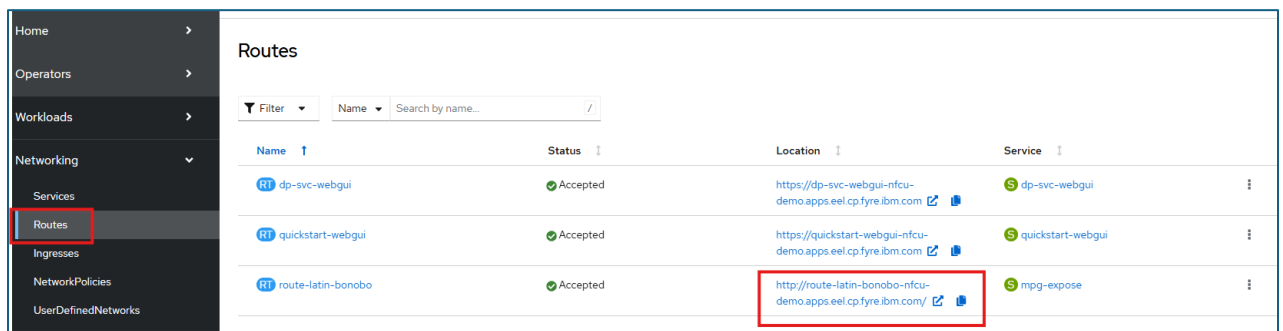


Routes				
Filter	Name	Status	Location	Service
	dp-svc-webgui	Accepted	https://dp-svc-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	dp-svc-webgui
	quickstart-webgui	Accepted	https://quickstart-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	quickstart-webgui
	route-latin-bonobo	Accepted	http://route-latin-bonobo-nfcu-demo.apps.eel.cp.fyre.ibm.com	mpg-expose

2. Log in with the lab credentials below.

Field	Value
Username	admin
Password	admin

3. Confirm that the default and demo domains are available.
4. Import [mpg-expose.zip](#) into the demo domain.
5. Select the Link below to Test the mpg-expose MPGW



Routes				
Filter	Name	Status	Location	Service
	dp-svc-webgui	Accepted	https://dp-svc-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	dp-svc-webgui
	quickstart-webgui	Accepted	https://quickstart-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	quickstart-webgui
	route-latin-bonobo	Accepted	http://route-latin-bonobo-nfcu-demo.apps.eel.cp.fyre.ibm.com	mpg-expose

6. Confirm that the service returns a JSON response.
7. Review the related OpenShift objects.

Command:

```
oc get pods  
oc get datapowerservice  
oc get route
```

Checkpoint

- Web UI login works.
- The default domain is up.
- The MPG route responds successfully.
- The base platform is ready for the Git-based extension lab.
- **Note – at this point if you delete the POD the MPGW imported into the DataPower Service will be deleted.**

Lab 8: Pull configuration from Git

Objective

Add a new DataPower domain that uses the GitOps feature to pull configuration from GitHub, then confirm the configuration is visible in the web UI and written back to the repository.

Part A: Create required resources

1. Confirm that you are in the `nfcu-demo` project.

Command:

```
oc project nfcu-demo
```

2. Create the ConfigMap from `pull-from-git.cfg`.

Command:

```
oc create configmap pull-from-git-cfg --from-file=pull-from-git.cfg -n nfcu-demo
```

3. Recreate the GitHub certificate secret to ensure the correct certificate file is being used.

Command:

```
oc delete secret github.com.cer -n nfcu-demo
oc create secret generic github.com.cer --from-file=github.com.cer -n nfcu-demo
```

4. Create the GitHub personal access token secret. Use a token with repository access.
5. IBM Team will provide PAT token.

Command:

```
oc create secret generic pat --from-literal=password=<YOUR_GITHUB_PAT_TOKEN> -n nfcu-demo
```

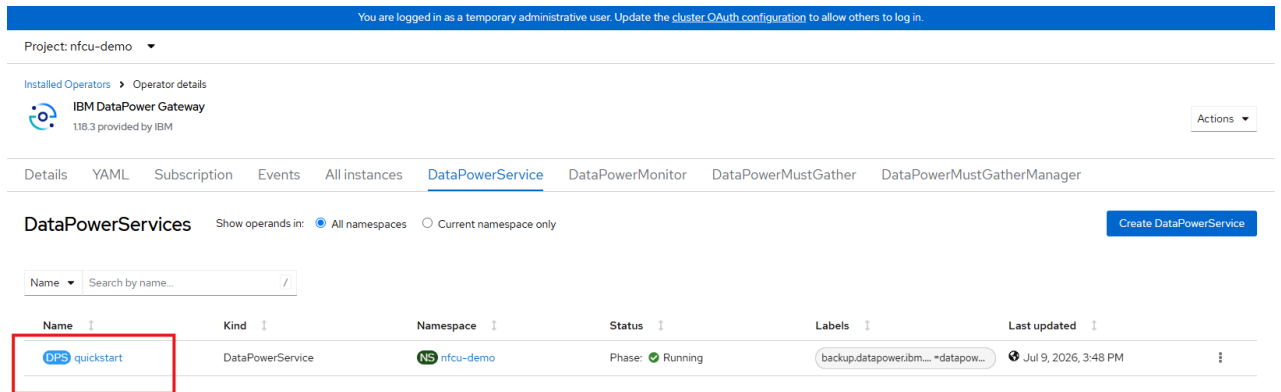
5. Verify the resources.

Command:

```
oc get configmap pull-from-git-cfg -n nfcu-demo
oc get secret github.com.cer -n nfcu-demo
oc get secret pat -n nfcu-demo
```

Part B: Update the DataPowerService

1. In the OpenShift console, open **Operators > Installed Operators > IBM DataPower Gateway > DataPowerService**.
2. Click to open the quickstart DataPowerService.



3. Download the current YAML definition (Download in the Bottom Corner).
4. It will should be named **datapowerservice-quickstart.yaml**
5. Open the (**datapowerservice-quickstart.yaml**) downloaded file in a text editor.
6. Download and open **pot-additionalDomainConfig.yaml** in a text editor from the lab repository.
7. Select All the contents from the **pot-additionalDomainConfig.yaml**
8. Replace the existing **domains**: section in the downloaded DataPowerService YAML (**datapowerservice-quickstart.yaml**) with the updated domain configuration.

Before:

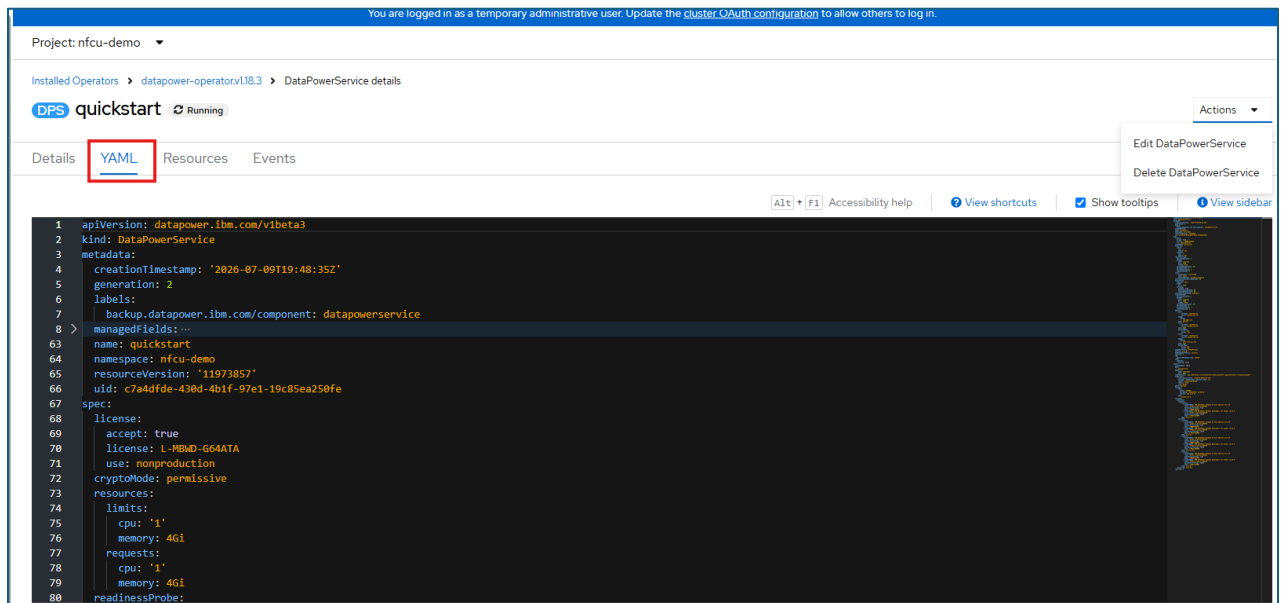
```
114 domains:
115   - certs:
116     - certType: sharedcerts
117       secret: github.com.cer
118     dpApp:
119       config:
120         - web-mgmt-cfg
121     name: default
122   - certs:
123     - certType: sharedcerts
124       secret: github.com.cer
125     name: demo
```

After:

```
114 domains:
115   - certs:
116     - certType: sharedcerts
117       secret: github.com.cer
118     dpApp:
119       config:
120         - web-mgmt-cfg
121     name: default
122   - certs:
123     - certType: sharedcerts
124       secret: github.com.cer
125     passwordMap:
126       - alias: pat
127         secret: pat
128     name: demo
129   - certs:
130     - certType: sharedcerts
131       secret: github.com.cer
132     dpApp:
133       config:
134         - pull-from-git-cfg
135     passwordMap:
136       - alias: pat
137         secret: pat
138     name: Test
```

9. Save the updated YAML (**datapowerservice-quickstart.yaml**).

10. In the OpenShift console select the **YAML** tab of the quickstart DataPowerService



11. Click **Edit DataPowerService**

12. Paste the updated YAML (**datapowerservice-quickstart.yaml**) with the additional configuration

13. Click Save

14. Click Reload

15. Optional Watch the pod restart and return to **Running**.

Command:

```
oc get pods -w
```

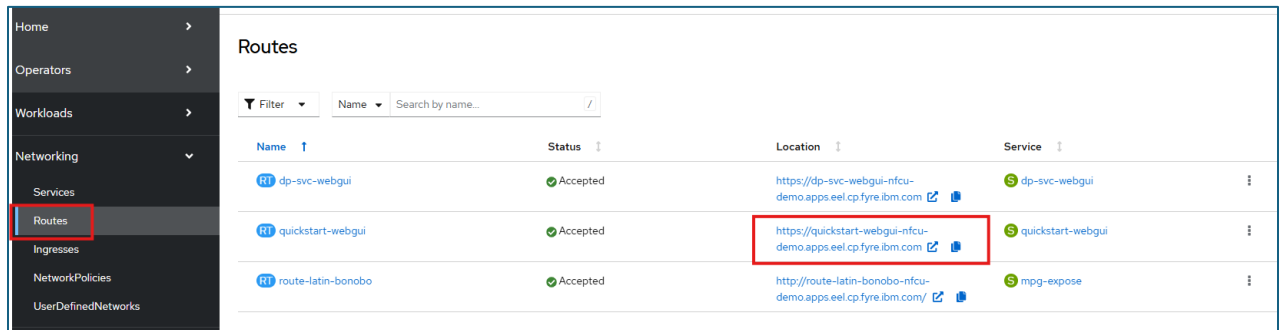
10. Confirm that the updated DataPowerService includes the new Test domain configuration.

Command:

```
oc get datapowerservice quickstart -o yaml
```

Part C: Validate in the web UI

1. Open the DataPower web UI using the quickstart-webgui route.



Routes				
Filter Name Search by name... /				
Name ↑	Status ↓	Location ↓	Service ↓	
dp-svc-webgui	Accepted	https://dp-svc-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	dp-svc-webgui	
quickstart-webgui	Accepted	https://quickstart-webgui-nfcu-demo.apps.eel.cp.fyre.ibm.com	quickstart-webgui	
route-latin-bonobo	Accepted	http://route-latin-bonobo-nfcu-demo.apps.eel.cp.fyre.ibm.com	mpg-expose	

2. Log in as **admin** with password **admin**.
3. Confirm that a new Test domain appears alongside default and demo.
4. Switch to the Test domain.
5. Search for the **GitOps** object and open it.
6. Confirm that the repository URL points to **https://github.com/innessrg/NFCU_MVP1_Source_Files.git**.
7. Open **Multi-Protocol Gateway** in the Test domain and review the services pulled from the repository.

Part D: Confirm GitHub

1. Open the repository [lab repository](#) in a browser.
2. Confirm that the expected DataPower domain configuration files are present.

Checkpoint

- pull-from-git-cfg exists.
- github.com.cer exists.
- pat exists.
- The DataPowerService was updated successfully.
- The pod restarted and returned to Running.
- The Test domain appears in the web UI.
- The GitOps object points to the correct repository.

- The expected services appear in the Test domain.
- Configuration is visible in the GitHub repository.
- Note- if you delete the POD the service will peresist

Troubleshooting

If the DataPowerService does not start correctly, use the following checks.

Check pod status and events

```
oc get pods -n nfcu-demo
oc describe pod quickstart-0 -n nfcu-demo
```

Review the **Events** section for pull errors, scheduling failures, or configuration issues.

Check DataPowerService status

```
oc describe datapowerservice quickstart -n nfcu-demo
```

Review the **Status** section for conditions or error messages.

Check operator logs

```
oc logs -n openshift-operators -l name=datapower-operator
```

Use the operator logs to identify reconciliation failures.

Check image pull configuration

If the pod is stuck in ImagePullBackOff or ErrImagePull, verify the entitlement secret and its service account link.

```
oc get secret ibm-entitlement-key -n nfcu-demo
oc get serviceaccount default -n nfcu-demo -o yaml | grep ibm-entitlement-key
```

Check for missing secrets or ConfigMaps

```
oc get secret,configmap -n nfcu-demo
```

Use this command to confirm that all referenced resources from Lab 4 exist.

Check namespace events

```
oc get events -n nfcu-demo --sort-by='.lastTimestamp'
```

Namespace events often provide the fastest explanation for a failed deployment.

Lab 9: Configure Persistent Storage for IBM DataPower on OpenShift

Objective

Configure persistent local storage for IBM DataPower on Red Hat OpenShift so the DataPower pod can use separate persistent mounts for `/config` and `/local` on the same worker node.

Lab Overview

This lab demonstrates how to use the Red Hat Local Storage Operator to provide node-local persistent storage for an IBM DataPower deployment on OpenShift. The design uses two partitions on the same secondary disk to model independent persistent volumes for the two DataPower storage paths listed below.

- `/opt/ibm/datapower/drouter/config`
- `/opt/ibm/datapower/drouter/local`

This separation is useful in a lab because it creates a clear one-to-one relationship between disk partitions, persistent volumes, persistent volume claims, and DataPower mount targets.

Important Design Note

This lab uses **node-local** storage, not shared cluster storage. The persistent volumes created from local partitions are tied to the worker node that owns the disk, so this design preserves data across pod recreation on that node but does not provide cross-node storage failover by itself.

Persistent storage is not a requirement for DataPower containers, nor is it generally best practice. Depends on the use case.. but generally speaking configuration should be managed and mounted via configMaps, secrets and repos, not mutated within the container and persisted to disk.

Any services/configuration that relies on DataPower's RAID capability would utilize persistent storage in k8s to back the file system

Learning Objectives

In this lab, you will:

- Retrieve node details to identify the target deployment node.
- Partition a raw secondary block device (`/dev/vdb`) on Red Hat Enterprise Linux CoreOS.
- Install and configure the Red Hat Local Storage Operator.
- Provision the `local-file-sc` storage class and bind node partitions to local persistent volumes.

- Deploy IBM DataPower with `deleteClaim: false` to preserve persistent state.
- Validate that configuration persists across pod recreation.

Prerequisites

Before starting, ensure the following requirements are met:

- Cluster-admin access to OpenShift is available.
- The `oc` CLI is installed and already logged in to the target cluster.
- A secondary block device is attached to the worker node and available as `/dev/vdb`.

Required DataPower images, entitlement credentials, secrets, and ConfigMaps from earlier labs are already present.

Files referenced in this lab include:

- [Datapower.yaml](#)
- [Datapower-lso.yaml](#)

Reference Documents

Use the following references during the lab:

- [Red Hat OpenShift Local Storage Operator documentation](#).
- [Local Storage Operator OLM deployment guidance](#).
- [IBM DataPower Operator storage documentation](#).
- [IBM DataPower initialization and ConfigMap reference documentation](#).
- [IBM architectural guidance for DataPower on OpenShift](#).

Exercise 1: Clean the Cluster Environment

Delete any existing DataPower custom resource before creating the new storage-backed deployment.

```
oc delete datapowerservice quickstart -n nfcu-demo --ignore-not-found
```

Exercise 2: Identify and Partition Host Worker Storage

Step 2.1: Locate the Target Node

Get the node where the DataPower pod is scheduled.

```
oc get pod quickstart-0 -n nfcu-demo -o wide
```

Locate the host name under the `NODE` column.

Because this lab uses local storage, the DataPower pod must run on the worker node that owns the local partitions and persistent volumes.

Step 2.2: Access the Host Shell

Start a debug session on the target worker node.

```
oc debug node/[UPDATE HOST FROM COMMAND IN STEP 2.1]
```

for example

```
oc debug node/worker2.eel.cp.fyre.ibm.com
```

Switch to the host filesystem.

```
chroot /host
```

Step 2.3: Partition `/dev/vdb`

Partition the disk using `fdisk`.

```
(
echo g
echo n
echo 1
echo
echo +100G
echo n
echo 2
echo
echo
```

```
echo w
)| fdisk /dev/vdb
```

Verify the partitions were created.

```
lsblk /dev/vdb
```

Expected result: /dev/vdb1 and /dev/vdb2 exist and are available for local PV creation.

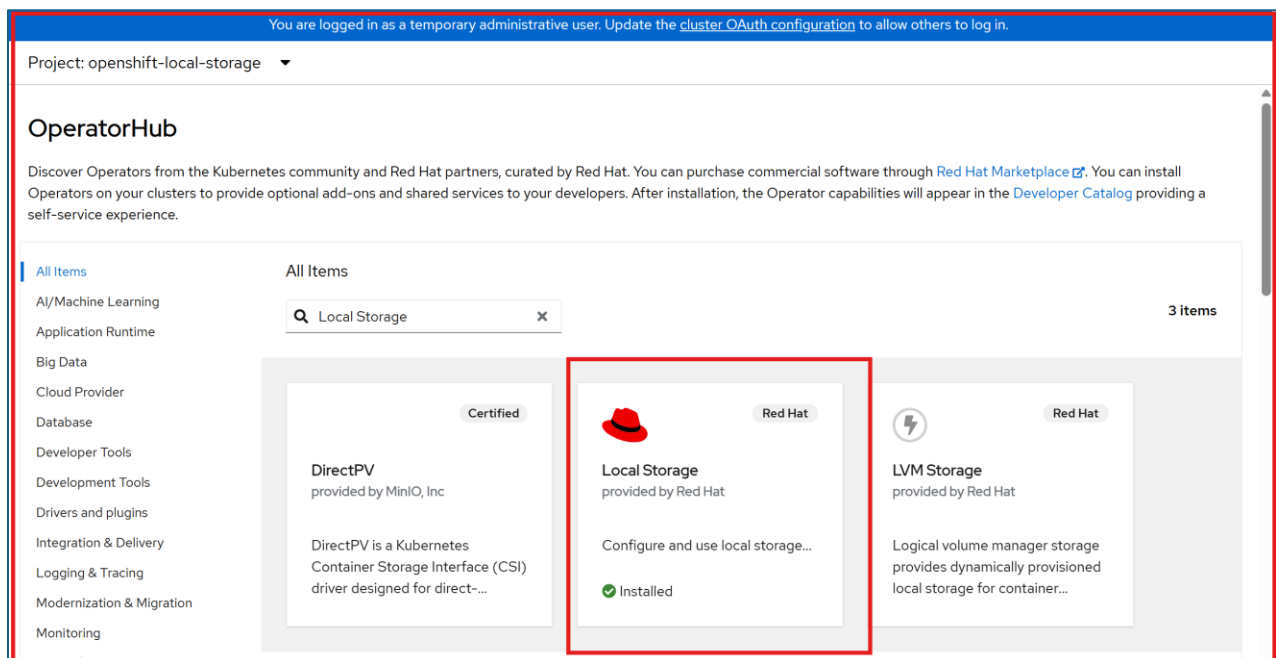
Exit the debug session.

```
exit
```

```
exit
```

Exercise 3: Install the Local Storage Operator

In the OpenShift web console, go to **Operators > OperatorHub** and search for **Local Storage**. Select the **Red Hat Local Storage Operator** tile and install it with the following settings.



- Update Channel: stable
- Installation Mode: A specific namespace on the cluster
- Installed Namespace: openshift-local-storage

- Update Approval: Automatic

Wait until the operator status shows **Succeeded** under **Operators > Installed Operators**.

Exercise 4: Provision the StorageClass and Persistent Volumes

Review `datapower-localvolume.yaml` from the repository before applying it. This manifest should define the local storage resources that discover and expose the partitions as local persistent volumes.

Apply the LocalVolume configuration.

```
oc apply -f datapower-localvolume.yaml
```

Verify that the storage class exists.

```
oc get sc local-file-sc
```

Verify the local persistent volumes.

```
oc get pv | grep local-file-sc
```

Ensure both entries are associated with `local-file-sc` and show as `Available` before moving forward.

Exercise 5: Deploy DataPower with Local Storage

Review `Datapower-lso.yaml` carefully before applying it. Confirm that the manifest contains two storage definitions, that the storage is mapped to the correct DataPower mount paths, that the storage class references are correct, and that `deleteClaim: false` is configured on the persistent storage entries.

Apply the DataPower manifest.

```
oc apply -f Datapower-lso.yaml
```

Exercise 6: Validate and Test Persistence

Step 6.1: Verify Pod and PVC Binding

Verify the persistent volume claims are bound.

```
oc get pvc -n nfcu-demo
```

Verify the DataPower pod is running.

```
oc get pods -n nfcu-demo
```

Step 6.2: Write and Commit Test Configurations

Open the DataPower WebGUI in your browser. Import your configuration assets, for example `pull-from-git-object.zip`, then click **Save Configuration** in the top-right corner of the WebGUI to commit changes to disk.

Step 6.3: Run the Pod Recreation Test

Delete the running DataPower pod.

```
oc delete pod quickstart-0 -n nfcu-demo
```

Watch the replacement pod come back.

```
oc get pods -n nfcu-demo -w
```

After the new `quickstart-0` pod is running, log back in and confirm that the imported configuration is still present.

Checkpoint

At this stage, confirm the following:

- Existing DataPower resources and stale storage mappings were cleaned up.
- The target worker node was identified.
- `/dev/vdb1` and `/dev/vdb2` were created successfully.
- The Local Storage Operator is installed and running.
- `local-file-sc` exists.
- Local persistent volumes are available.
- DataPower deployed successfully with `deleteClaim: false`.
- PVCs are bound.
- The WebGUI is reachable.
- Imported configuration persists after pod recreation.

Troubleshooting: DataPowerService Failed to Create

If the DataPowerService does not come up, use the following checks.

1. Check pod status and events

```
oc get pods -n nfcu-demo
oc describe pod quickstart-0 -n nfcu-demo
```

Review the Events section at the bottom of the describe output for image pull errors, scheduling failures, and configuration problems.

2. Check the DataPowerService resource status

```
oc describe datapowerservice quickstart -n nfcu-demo
```

Review any status conditions or error messages under Status.

3. Check the operator logs

```
oc logs -n openshift-operators -l name=datapower-operator
```

The operator is responsible for creating the pod, so reconciliation failures often appear here first.

4. Check for image pull errors

If the pod is stuck in ImagePullBackOff or ErrImagePull, verify that the IBM entitlement secret exists and is linked correctly.

```
oc get secret ibm-entitlement-key -n nfcu-demo
oc get serviceaccount default -n nfcu-demo -o yaml | grep ibm-entitlement-key
```

5. Check for missing secrets or ConfigMaps

If the pod fails with a mount error, verify that all referenced secrets and ConfigMaps from earlier labs are present.

```
oc get secret,configmap -n nfcu-demo
```

6. Review namespace events

```
oc get events -n nfcu-demo --sort-by='.lastTimestamp'
```

This command provides a useful event timeline and is often the quickest way to identify the immediate cause of a failed deployment.

Lab Notes for Production Design

This lab is intentionally designed for visibility and learning rather than for a production-ready storage architecture. In production, teams often use resilient shared storage or platform-native block storage instead of node-local partitions, but the separation of persistent DataPower storage concerns remains a useful design principle.